

Ejemplos de referencia · Taller Despacho reactivo

Patrones mínimos que el taller exige, en el dominio de **despachos**. Cada uno viene con el antipatrón que el evaluador busca primero. El código es de referencia: hay que adaptarlo, no pegarlo.

1. Reserva atómica de cupo con RETURNING

El invariante `cupo_kg >= 0` lo garantiza la base, no el código.

```
// CupoService.java
@Service
public class CupoService {

    private final DatabaseClient db;

    public CupoService(DatabaseClient db) { this.db = db; }

    /** Descuenta cupo solo si alcanza. Devuelve vacío si no alcanzó: un solo viaje, sin carrera. */
    public Mono<Vehiculo> reservar(long vehiculoId, int pesoKg) {
        return db.sql("""
            UPDATE vehiculo
            SET cupo_kg = cupo_kg - :peso, reservado_kg = reservado_kg + :peso
            WHERE id = :id AND cupo_kg >= :peso
            RETURNING id, placa, cupo_kg, reservado_kg, ciudad
            """)
            .bind("id", vehiculoId).bind("peso", pesoKg)
            .map((row, meta) -> new Vehiculo(
                row.get("id", Long.class),
                row.get("placa", String.class),
                row.get("cupo_kg", Integer.class),
                row.get("reservado_kg", Integer.class),
                row.get("ciudad", String.class)))
            .one()
            .switchIfEmpty(Mono.error(new CupoInsuficienteException(vehiculoId, pesoKg)));
    }

    /** Compensación: devuelve el cupo. Sin condición: siempre debe poder revertirse. */
    public Mono<Void> liberar(long vehiculoId, int pesoKg) {
        return db.sql("UPDATE vehiculo SET cupo_kg = cupo_kg + :peso, reservado_kg = reservado_kg - :peso WHERE id = :id")
            .bind("id", vehiculoId).bind("peso", pesoKg)
            .fetch().rowsUpdated().then();
    }
}
```

Antipatrón — lee, decide y escribe (dos despachos concurrentes dejan cupo negativo):

```
// MAL
return repo.findById(id)
    .filter(v -> v.cupoKg() >= peso) // decisión en la app
    .flatMap(v -> repo.save(v.restar(peso))); // otro hilo ya escribió entre medias
```

2. Saga de compensación

```
// AsignacionSaga.java
public Mono<List<Reserva>> reservarTodo(List<Paquete> paquetes) {
    List<Reserva> hechas = new CopyOnWriteArrayList<>();
    return Flux.fromIterable(paquetes)
        .concatMap(p -> cupo.reservar(p.vehiculoId(), p.pesoKg()) // concatMap: orden y rastro
            .doOnNext(v -> hechas.add(new Reserva(p.vehiculoId(), p.pesoKg())))
            .then(Mono.just(List.copyOf(hechas)))
            .onErrorResume(e -> compensar(hechas).then(Mono.error(e)))) // revierte y propaga el original
    }

    private Mono<Void> compensar(List<Reserva> hechas) {
        return Flux.fromIterable(hechas)
            .concatMap(r -> cupo.liberar(r.vehiculoId(), r.pesoKg())
                .doOnNext(e -> { // la compensación nunca debe tumbar el flujo
                    log.error("Compensación falló para {}: {}", r.vehiculoId(), e.toString());
                    return Mono.empty();
                })
            )
            .then();
    }
}
```

3. Tres externos en paralelo + resiliencia por servicio

```
// TransportistaClient.java
public Mono<Tarifa> tarifa(String ciudad) {
    return http.get().uri("/external/tarifa/{c}", ciudad)
        .retrieve().bodyToMono(Tarifa.class)
        .retryWhen(ReactiveSupport.backoff(3, Duration.ofMillis(200))) // solo transitorios
        .onErrorResume(e -> catalogo.tarifaBase(ciudad)); // fallback, no 500
}

public Mono<Integer> riesgo(String ciudad) {
    return http.get().uri("/external/riesgo/{c}", ciudad)
        .retrieve().bodyToMono(Integer.class)
        .timeout(Duration.ofMillis(800))
        .onErrorReturn(30); // score por defecto
}

public Mono<Ventana> ventana(String ciudad) {
    return cachePorCiudad.computeIfAbsent(ciudad, c ->
        http.get().uri("/external/clima/{c}", c)
        .retrieve().bodyToMono(Ventana.class)
        .cache(Duration.ofMinutes(10))); // hot: una llamada por 10 min
}
```

```
// DespachoService.java - zip: las tres salen a la vez
return Mono.zip(externos.tarifa(ciudad), externos.ventana(ciudad), externos.riesgo(ciudad))
    .flatMap(t -> {
        int score = t.getT3();
        if (score > 80) return Mono.error(new ZonaRiesgosaException(ciudad, score));
        return calcularTotales(despacho, t.getT1(), t.getT2());
    });
```

Antipatrón — `zip` que no paraleliza nada:

```
// MAL: cada paso espera al anterior, el zip llega tarde
return externos.tarifa(c)
    .flatMap(tar -> externos.ventana(c)
        .flatMap(ven -> externos.riesgo(c).map(r -> new Trio(tar, ven, r))));
```

4. Transacción R2DBC acotada

```
// DespachoService.java
private final TransactionalOperator tx;

private Mono<Despacho> persistir(Despacho d, List<Paquete> paquetes) {
    return despachoRepo.save(d.conEstado(ASIGNADO).expiraEn(Instant.now().plus(Duration.ofMinutes(15))))
        .flatMap(guardado -> paqueteRepo
            .saveAll(paquetes.stream().map(p -> p.conDespacho(guardado.id())).toList())
            .then(Mono.just(guardado)))
        .as(tx::transactional); // la transacción cubre SOLO escrituras a la BD
}
```

Regla: ninguna llamada HTTP dentro de `tx::transactional`. La conexión queda tomada y un timeout externo deja la transacción abierta. Los externos van antes; el cupo se revierte con saga.

5. Bus de eventos hot + SSE que cierra

```
// EventBus.java
private final Sinks.Many<EventoDespacho> eventos = Sinks.many().replay().limit(200);

public void publicar(EventoDespacho e) { eventos.emitNext(e, Sinks.EmitFailureHandler.FAIL_FAST); }
public Flux<EventoDespacho> stream() { return eventos.asFlux(); }
```

```
// DespachoController.java - SSE de un despacho: filtra y cierra en estado terminal
@GetMapping(value = "/api/despachos/{id}/events", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<ServerSentEvent<EventoDespacho>> eventos(@PathVariable long id) {
    return bus.stream()
        .filter(e -> e.despachoId() == id)
        .takeUntil(e -> e.estado().esTerminal()) // cierra: no deja la conexión colgada
        .map(e -> ServerSentEvent.builder(e).event(e.estado().name()).build())
        .doOnCancel(() -> log.info("cliente cerró el SSE del despacho {}", id));
}
```

```
// TableroController.java - tablero global: hot compartido + último valor
@GetMapping(value = "/api/ops/tablero", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<EventoTablero> tablero() {
    return tableroService.stream() // publish().refCount(1) adentro: una fuente para todos
        .onBackpressureLatest() // consumidor lento: le llega el último, no una cola
        .distinctUntilChanged();
}
```

Antipatrón — cold por suscriptor: si `stream()` devuelve `Flux.interval(...).flatMap(repo::...)` sin `publish().refCount()`, cada operador conectado abre su propia consulta y ve datos distintos.

6. Reporte con backpressure y agregación

```
// ReporteService.java
public Flux<TotalCiudad> porCiudad() {
    return paqueteRepo.findAllConCiudad()
        .limitRate(256) // pide de a 256, no se traga la tabla
        .groupBy(PaqueteCiudad::ciudad)
        .flatMap(g -> g.reduce(TotalCiudad.vacio(g.key()), TotalCiudad::sumar))
        .collectSortedList(comparing(TotalCiudad::kilos).reversed())
        .flatMapIterable(1 -> 1);
}

/** Acumulado en vivo: scan mantiene el estado entre eventos. */
public Flux<TotalCiudad> stream() {
    return bus.stream()
        .scan(TotalCiudad.vacio("TODAS"), (acc, e) -> acc.sumar(e.kilos()))
        .sample(Duration.ofSeconds(1));
}
```

7. trazaId por Reactor Context

```
// TrazaWebFilter.java
@Component
public class TrazaWebFilter implements WebFilter {
    public static final String KEY = "trazaId";

    @Override
    public Mono<Void> filter(ServerWebExchange ex, WebFilterChain chain) {
        String id = Optional.ofNullable(ex.getRequest().getHeaders().getFirst("X-Traza-Id"))
            .orElseGet(() -> UUID.randomUUID().toString().substring(0, 8));
        ex.getResponse().getHeaders().add("X-Traza-Id", id);
        return chain.filter(ex).contextWrite(ctx -> ctx.put(KEY, id));
    }
}
```

```
// Consumo, en cualquier capa, sin recibirlo como parámetro
return Mono.deferContextual(ctx -> {
    String traza = ctx.getOrDefault(TrazaWebFilter.KEY, "n/a");
    log.info("{} reservando cupo del vehículo {}", traza, vehiculoId);
    return cupo.reservar(vehiculoId, peso);
});
```

Antipatrón: `ThreadLocal` o MDC plano. En WebFlux la cadena salta de hilo y el valor se pierde.

8. Job de expiración con ciclo de vida

```
// ExpiracionJob.java
@Component
public class ExpiracionJob {

    private Disposable suscripcion;

    @PostConstruct
    void arrancar() {
        suscripcion = Flux.interval(Duration.ofSeconds(30))
            .onBackpressureDrop(t -> log.warn("tick {} descartado: el anterior sigue corriendo", t))
            .concatMap(t -> expirarVencidas() // concatMap: nunca dos pasadas a la vez
                .onErrorResume(e -> { // un error no mata el intervalo
                    log.error("fallo expirando: {}", e.toString());
                    return Mono.empty();
                })
            )
            .subscribe();
    }

    @PreDestroy
    void parar() { if (suscripcion != null) suscripcion.dispose(); }
}
```

9. Pruebas

```
// StepVerifier: caso feliz y error
@Test
void reservaDescuentaCupo() {
    StepVerifier.create(cupo.reservar(1L, 100))
        .assertNext(v -> assertThat(v.cupoKg()).isEqualTo(400))
        .verifyComplete();
}

@Test
void sinCupoLanzaExcepcion() {
    StepVerifier.create(cupo.reservar(1L, 9_999))
        .expectError(CupoInsuficienteException.class)
        .verify();
}
```

```
// withVirtualTime: prueba el retry/timeout sin esperar de verdad
@Test
void reintentaTresVecesYUsaFallback() {
    StepVerifier.withVirtualTime(() -> client.tarifa("BOG"))
        .thenAwait(Duration.ofSeconds(10))
        .assertNext(t -> assertThat(t.origen()).isEqualTo("CATALOGO"))
        .verifyComplete();
}
```

```
// TestPublisher: controla la fuente a mano
@Test
void sseCierraEnEstadoTerminal() {
    TestPublisher<EventoDespacho> fuente = TestPublisher.create();
    StepVerifier.create(Flux.from(fuente).takeUntil(e -> e.estado().esTerminal()))
        .then(() -> fuente.next(evento(ASIGNADO)))
        .expectNextCount(1)
        .then(() -> fuente.next(evento(ENTREGADO)))
        .expectNextCount(1)
        .verifyComplete(); // cerró solo, sin que la fuente complete
}
```

```
// WebTestClient: E2E del contrato
@Test
void riesgoAltoDevuelve422YLiberaCupo() {
    web.put().uri("/external/simulator").bodyValue(new SimulatorConfig(0, 0, 95)).exchange().expectStatus().isOk();

    web.post().uri("/api/despachos").header("X-Traza-Id", "t1").bodyValue(request)
        .exchange()
        .expectStatus().isEqualTo(422)
        .expectBody().jsonPath("$.codigo").isEqualTo("ZONA_RIESGOSA")
        .jsonPath("$.trazaId").isEqualTo("t1");

    StepVerifier.create(vehiculoRepo.findById(1L))
        .assertNext(v -> assertThat(v.cupoKg()).isEqualTo(500)) // se liberó
        .verifyComplete();
}
```

```
// Concurrencia: el invariante bajo carga
@Test
void veinteDespachosConcurrentesNoDejanCupoNegativo() {
    StepVerifier.create(
        Flux.range(0, 20)
            .flatMap(i -> cupo.reservar(1L, 50).onErrorResume(e -> Mono.empty()), 20)
            .then(vehiculoRepo.findById(1L))
            .assertNext(v -> assertThat(v.cupoKg()).isGreaterThanOrEqualTo(0))
            .verifyComplete();
    )
}
```

10. Antipatrones que el evaluador busca primero

Antipatrón	Por qué está mal	En su lugar
<code>repo.findById(id).block()</code>	Bloquea el event loop: con 4 hilos, 4 peticiones congelan la app	Encadenar con <code>flatMap</code>
Cadena armada y nunca suscrita	El código no se ejecuta y el test pasa igual	Devolver el <code>Mono/Flux</code> o <code>.subscribe()</code> en el job
<code>Thread.sleep</code> en un test	Lo vuelve lento y frágil	<code>StepVerifier.withVirtualTime</code>
<code>@Transactional</code> sobre método reactivo	No abarca la cadena: no hay transacción	<code>TransactionalOperator</code> + <code>.as(tx::transactional)</code>
<code>retryWhen</code> sin filter	Reintenta un 404 o un 400 tres veces	<code>Retry.backoff(...).filter(::isTransient)</code>
<code>onErrorResume(e -> Mono.empty())</code> global	Se traga errores de dominio y el cliente recibe 200 vacío	Filtrar por tipo y mapear a excepción de dominio
Sinks con un solo suscriptor asumido	Con dos clientes se pierden eventos o se duplican	<code>multicast()</code> / <code>replay()</code> según el caso
Buffer infinito en el tablero	Un cliente lento tumba la app por memoria	<code>onBackpressureLatest</code>
<code>trazad</code> como parámetro en 8 métodos	Ensucia toda la API por un dato transversal	Reactor <code>Context</code>
<code>subscribe()</code> dentro de un <code>map</code>	Fuga de suscripción, errores invisibles	<code>flatMap</code>